

Electronics Domain

Simulator option: You do not need to own physical hardware to attempt these tasks. You may build and test your circuit on Wokwi (a free, browser-based Arduino/ESP32 simulator) and submit a link to your project. If you have the physical components, you may instead submit a short video of the circuit running. Either submission format is equally acceptable.

Task 1 — Redundant Sensor Voting

Components: 2× potentiometer (treated as two independent sensors reading the same physical quantity), 1× servo motor, 1× fault LED, Arduino Nano.

Wiring: Potentiometers to two separate analog pins (e.g. A0, A1). Servo signal pin and fault LED to pins of your choice — state which pins you used in your submission.

Behavior: Continuously read both potentiometers (0–1023) and independently map each to a 0–180° range.

- When the two mapped values are within a tolerance you define (state and justify this threshold), set the servo angle to the mean of the two mapped values, and keep the fault LED off.
- When the two mapped values diverge beyond that threshold, light the fault LED. The servo must hold its last valid angle — do not update it based on the current, untrustworthy readings.
- Your logic must avoid flicker at the boundary: the fault condition should not rapidly toggle on/off if the readings are hovering right around your threshold. Decide how you handle this (e.g. separate trigger/clear thresholds) and justify your approach.

Deliverable:

- Wokwi project link (preferred — both potentiometers can be driven programmatically via Wokwi's Automation Scenarios for testing) or video of physical hardware
- Code
- Design rationale (150–250 words): threshold value and why, how "hold last position" is implemented, how boundary flicker was prevented.

Task 2 — Simon Says with Heartbeat

Components: 4× LED, 4× button, 1× additional "heartbeat" LED, Arduino Nano.

Behavior: Implement the classic Simon Says game:

- The device plays a randomly generated, growing sequence of LED flashes.
- The player must repeat the sequence back using the matching buttons, in order.

- A correct repeat grows the sequence by one step; a wrong input resets the game.
- Sequence playback speed must increase as the sequence grows — your ramp/formula, justified in your writeup.
- The heartbeat LED must blink at a constant, fixed interval continuously — through sequence playback, through the input-wait window, through everything — never pausing or stuttering regardless of what the game is doing at that moment.

Deliverable:

- Wokwi project link (fully testable visually in-browser, no special tooling needed) or video of physical hardware
- Code
- Design rationale (150–250 words): your speed-ramp formula and how you kept the heartbeat LED independent of game state.

Note: If the heartbeat LED visibly stutters or pauses at any point (during a sequence flash, while waiting for input, etc.), the non-blocking requirement has not been met, regardless of whether the core game logic otherwise works.

Task 3 — MPU6050 KiCad Schematic (Bonus)

Reference: MPU6050 electrical datasheet attached (see [MPU6050_datasheet](#)).

Requirement — schematic: Design the schematic (schematic capture only — no PCB layout required for this part) for a breakout module built around the MPU6050 IC:

- MPU6050 symbol with all pins correctly connected/labeled.
- Decoupling capacitor on VDD, value and placement per the datasheet's recommendation.
- I2C pull-up resistors on SDA/SCL, sized per datasheet guidance.
- AD0 pin tied to a defined logic level — state which I2C address this selects, and why.
- INT pin routed to a labeled header pin.
- A labeled 4-pin header (VCC, GND, SDA, SCL — plus INT if you choose) as the board's external interface.
- Schematic should pass an Electrical Rules Check (ERC) cleanly — no unconnected pins, no conflicting outputs.

Requirement — PCB layout (optional, extra credit): Route an actual PCB layout from your schematic. Not required to attempt Task 3 at all, but scored as bonus-on-bonus if submitted.

Deliverable:

- KiCad project (schematic files, or an exported schematic PDF at minimum; PCB layout files if you attempt that part)

- A short rationale (200–350 words): your decoupling/pull-up value choices and why, and why you tied AD0 the way you did.
-